

thu225187153c-dependency-parsing

December 8, 2025

Họ và tên: Đỗ Thảo Giang

Mã sinh viên: 22001253

1 Cài đặt thư viện

```
[ ]: !pip install -U spacy
```

Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)

Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)

Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)

Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)

Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)

Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)

Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)

Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)

Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)

Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)

Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)

Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.20.0)

Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (4.67.1)

Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)

Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)

Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)

Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.4)

Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)

Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2025.11.12)

Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (1.3.3)

Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (0.1.5)

Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0->spacy) (8.3.1)

Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (0.23.0)

Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (7.5.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy) (3.0.3)

Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages

```
(from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy) (2.0.1)
```

```
[ ]: import spacy
     from spacy import displacy
```

```
[ ]: !python -m spacy download en_core_web_md
```

Collecting en-core-web-md==3.8.0

Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_md-3.8.0/en_core_web_md-3.8.0-py3-none-any.whl (33.5 MB)

33.5/33.5 MB

56.6 MB/s eta 0:00:00

Installing collected packages: en-core-web-md

Successfully installed en-core-web-md-3.8.0

Download and installation successful

You can now load the package via `spacy.load('en_core_web_md')`

Restart to reload dependencies

If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

2 Phân tích câu và Trực quan hóa

2.1 Tải mô hình và phân tích câu

```
[ ]: import spacy
     from spacy import displacy
```

```
[ ]: nlp = spacy.load("en_core_web_md")
```

```
[ ]: text = "The quick brown fox jumps over the lazy dog."
```

```
[ ]: doc = nlp(text)
```

2.2 Trực quan hóa cây phụ thuộc

```
[ ]: # Visualize the dependency tree using displacy.render for Colab
     displacy.render(doc, style="dep", jupyter=True, options={"compact": True})
```

<IPython.core.display.HTML object>

2.3 Câu hỏi

1. Từ gốc (ROOT) của câu là: jumps
2. Từ 'jumps' có các từ phụ thuộc sau:

- ‘fox’ với quan hệ: ‘nsubj’
 - ‘over’ với quan hệ: ‘prep’
3. Từ ‘fox’ là head của các từ sau:
- ‘The’ (quan hệ: ‘det’)
 - ‘quick’ (quan hệ: ‘amod’)
 - ‘brown’ (quan hệ: ‘amod’)

3 Truy cập các thành phần trong cây phụ thuộc

```
[ ]: # Lấy một câu khác để phân tích
text = "Apple is looking at buying U.K. startup for $1 billion."
doc = nlp(text)
# In ra thông tin của từng token
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | _
↳{'CHILDREN'}")
print("-" * 70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{token.text:<12} | {token.dep_:<10} | {token.head.text:<12} | _
↳{token.head.pos_:<8} | {str(children)}")
```

TEXT	DEP	HEAD TEXT	HEAD POS	CHILDREN

Apple	nsubj	looking	VERB	[]
is	aux	looking	VERB	[]
looking	ROOT	looking	VERB	['Apple', 'is', 'at', '.']
at	prep	looking	VERB	['buying']
buying	pcomp	at	ADP	['startup']
U.K.	compound	startup	NOUN	[]
startup	dobj	buying	VERB	['U.K.', 'for']
for	prep	startup	NOUN	['billion']
\$	quantmod	billion	NUM	[]
1	compound	billion	NUM	[]
billion	pobj	for	ADP	['\$', '1']
.	punct	looking	VERB	[]

4 Duyệt cây phụ thuộc để trích xuất thông tin

4.1 Bài toán: Tìm chủ ngữ và tân ngữ của một động từ

```
[ ]: text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)
for token in doc:
```

```

# Chỉ tìm các động từ
if token.pos_ == "VERB":
    verb = token.text
    subject = ""
    obj = ""
    # Tìm chủ ngữ (nsubj) và tân ngữ (dobj) trong các con của động từ
    for child in token.children:
        if child.dep_ == "nsubj":
            subject = child.text
        if child.dep_ == "dobj":
            obj = child.text
    if subject and obj:
        print(f"Found Triplet: ({subject}, {verb}, {obj})")

```

Found Triplet: (cat, chased, mouse)

Found Triplet: (dog, watched, them)

4.2 Bài toán: Tìm các tính từ bổ nghĩa cho một danh từ

```

[ ]: text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)
for token in doc:
    # Chỉ tìm các danh từ
    if token.pos_ == "NOUN":
        adjectives = []
        # Tìm các tính từ bổ nghĩa (amod) trong các con của danh từ
        for child in token.children:
            if child.dep_ == "amod":
                adjectives.append(child.text)
        if adjectives:
            print(f"Danh từ '{token.text}' được bổ nghĩa bởi các tính từ: ␣
↪{adjectives}")

```

Danh từ 'cat' được bổ nghĩa bởi các tính từ: ['big', 'fluffy', 'white']

Danh từ 'mat' được bổ nghĩa bởi các tính từ: ['warm']

5 Bài tập tự luyện

5.1 Bài 1: Tìm động từ chính của câu

```

[ ]: def find_main_verb(doc):
    """Tìm và trả về token là động từ chính (ROOT) của câu."""
    for token in doc:
        if token.dep_ == "ROOT" and token.pos_ == "VERB":
            return token
    return None

```

5.1.1 Test

```
[ ]: text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)
main_verb = find_main_verb(doc)

if main_verb:
    print(f"Động từ chính của câu là: '{main_verb.text}'")
else:
    print("Không tìm thấy động từ chính.")
```

Động từ chính của câu là: 'jumps'

```
[ ]: text_2 = "Apple is looking at buying U.K. startup for $1 billion."
doc_2 = nlp(text_2)
main_verb_2 = find_main_verb(doc_2)

if main_verb_2:
    print(f"Động từ chính của câu là: '{main_verb_2.text}'")
else:
    print("Không tìm thấy động từ chính.")
```

Động từ chính của câu là: 'looking'

5.2 Bài 2: Trích xuất các cụm danh từ (Noun Chunks)

```
[ ]: def extract_noun_chunks_manual(doc):
    """
    Tự viết hàm để trích xuất các cụm danh từ (noun chunks) từ đối tượng Doc
    của spaCy.
    Một cụm danh từ đơn giản được định nghĩa là một danh từ và tất cả các từ bổ
    nghĩa trực tiếp cho nó
    (determiner, adjectival modifier, compound, numerical modifier, possessive).
    """
    manual_noun_chunks = []
    extracted_chunks_set = set() # Sử dụng set để theo dõi các cụm đã thêm và
    tránh trùng lặp

    for token in doc:
        # Chúng ta tìm các danh từ hoặc danh từ riêng làm đầu (head) của cụm
        danh từ
        if token.pos_ in ["NOUN", "PROPN"]:
            chunk_elements = [token] # Bắt đầu với chính danh từ đó

            # Duyệt qua các từ con (children) của danh từ để tìm các từ bổ
            nghĩa trực tiếp
            for child in token.children:
```

```

        # Các quan hệ phụ thuộc thường gặp trong cụm danh từ
        if child.dep_ in ["det", "amod", "compound", "nummod", "poss"]:
            chunk_elements.append(child)

        # Sắp xếp các token theo vị trí của chúng trong câu để tạo cụm đúng
        ↳thứ tự từ trái sang phải
        if chunk_elements:
            chunk_elements.sort(key=lambda t: t.i)
            # Nối văn bản của các token để tạo thành chuỗi cụm danh từ
            chunk_string = " ".join([t.text for t in chunk_elements])

            # Thêm vào set để đảm bảo tính duy nhất
            extracted_chunks_set.add(chunk_string)

    # Chuyển đổi set thành list và trả về
    # Lưu ý: Việc chuyển đổi từ set sang list sẽ không đảm bảo thứ tự.
    # Nếu cần giữ thứ tự xuất hiện, cần một logic phức tạp hơn để lọc trùng lặp.
    return list(extracted_chunks_set)

```

5.2.1 Test

```

[ ]: text1 = "The quick brown fox jumps over the lazy dog."
     doc1 = nlp(text1)
     print(f"Original text: '{text1}'")
     print(f"spaCy's noun_chunks: {[chunk.text for chunk in doc1.noun_chunks]}")
     print(f"Manual noun_chunks: {extract_noun_chunks_manual(doc1)}\n")

```

```

Original text: 'The quick brown fox jumps over the lazy dog.'
spaCy's noun_chunks: ['The quick brown fox', 'the lazy dog']
Manual noun_chunks: ['The quick brown fox', 'the lazy dog']

```

```

[ ]: text2 = "Apple is looking at buying U.K. startup for $1 billion."
     doc2 = nlp(text2)
     print(f"Original text: '{text2}'")
     print(f"spaCy's noun_chunks: {[chunk.text for chunk in doc2.noun_chunks]}")
     print(f"Manual noun_chunks: {extract_noun_chunks_manual(doc2)}\n")

```

```

Original text: 'Apple is looking at buying U.K. startup for $1 billion.'
spaCy's noun_chunks: ['Apple', 'U.K. startup']
Manual noun_chunks: ['U.K.', 'U.K. startup', 'Apple']

```

```

[ ]: text3 = "a red car door"
     doc3 = nlp(text3)
     print(f"Original text: '{text3}'")
     print(f"spaCy's noun_chunks: {[chunk.text for chunk in doc3.noun_chunks]}")

```

```
print(f"Manual noun_chunks: {extract_noun_chunks_manual(doc3)}\n")
print("--- Test 3 (Compound Noun) ---")
```

Original text: 'a red car door'
spaCy's noun_chunks: ['a red car door']
Manual noun_chunks: ['a car door', 'red car']

--- Test 3 (Compound Noun) ---

```
[ ]: text4 = "a very beautiful day"
doc4 = nlp(text4)
print(f"Original text: '{text4}'")
print(f"spaCy's noun_chunks: {[chunk.text for chunk in doc4.noun_chunks]}")
print(f"Manual noun_chunks: {extract_noun_chunks_manual(doc4)}\n")
```

Original text: 'a very beautiful day'
spaCy's noun_chunks: ['a very beautiful day']
Manual noun_chunks: ['a beautiful day']

5.3 Bài 3: Tìm đường đi ngắn nhất trong cây

```
[ ]: def get_path_to_root(token):
    """
    Tìm đường đi từ một token bất kỳ lên đến gốc (ROOT) của cây phụ thuộc.
    Trả về một danh sách các token trên đường đi, bắt đầu từ token đầu vào.
    """
    path = []
    current_token = token
    while current_token != current_token.head:
        path.append(current_token)
        current_token = current_token.head
    path.append(current_token) # Thêm token ROOT vào cuối
    return path
```

5.3.1 Test

```
[ ]: text = "The quick brown fox jumps over the lazy dog."
doc = nlp(text)
print(f"Original text: '{text}'\n")
```

Original text: 'The quick brown fox jumps over the lazy dog.'

```
[ ]: fox_token = doc[3] # 'fox' là token thứ 3 (index 3)
path_fox = get_path_to_root(fox_token)
print(f"Đường đi từ '{fox_token.text}' đến ROOT: {[t.text for t in path_fox]}")
```



```
print(f"Quan hệ phụ thuộc trên đường đi: {[t.dep_ for t in path_fox]}\n")
```

Đường đi từ 'fox' đến ROOT: ['fox', 'jumps']

Quan hệ phụ thuộc trên đường đi: ['nsubj', 'ROOT']

```
[ ]: lazy_token = doc[7] # 'lazy' là token thứ 7 (index 7)
path_lazy = get_path_to_root(lazy_token)
print(f"Đường đi từ '{lazy_token.text}' đến ROOT: {[t.text for t in_
↳path_lazy]}\n")
print(f"Quan hệ phụ thuộc trên đường đi: {[t.dep_ for t in path_lazy]}\n")
```

Đường đi từ 'lazy' đến ROOT: ['lazy', 'dog', 'over', 'jumps']

Quan hệ phụ thuộc trên đường đi: ['amod', 'pobj', 'prep', 'ROOT']

```
[ ]: jumps_token = doc[4] # 'jumps' là token thứ 4 (index 4) và là ROOT
path_jumps = get_path_to_root(jumps_token)
print(f"Đường đi từ '{jumps_token.text}' (ROOT) đến ROOT: {[t.text for t in_
↳path_jumps]}\n")
print(f"Quan hệ phụ thuộc trên đường đi: {[t.dep_ for t in path_jumps]}\n")
```

Đường đi từ 'jumps' (ROOT) đến ROOT: ['jumps']

Quan hệ phụ thuộc trên đường đi: ['ROOT']